PROYECTO FINAL DE LA UC 1 (SISTEMAS DISTRIBUIDOS)

Desarrollo de Billetera Móvil Distribuida con Concurrencia y Persistencia

Sistemas Distribuidos / Tecnologías de la Información

Tema Central: Gestión de hilos, WebSockets, APIs REST y bases de datos locales (Room).

Descripción General del Sistema

Estimados estudiantes, desarrollar el cliente móvil (Android/Kotlin) de una Billetera Virtual. La aplicación no funcionará de manera aislada, sino que actuará como un nodo cliente dentro de un ecosistema distribuido.

La práctica evalúa la capacidad del estudiante para orquestar múltiples hilos de ejecución de forma segura: el hilo principal (Main Thread) para la interfaz gráfica, y los hilos de fondo (Background Threads) para la persistencia local (Room SQLite), las peticiones síncronas (REST) y la escucha asíncrona continua (WebSockets).

2. Funcionalidades Requeridas (Lógica de Negocio)

La aplicación debe interactuar con el backend mediante peticiones REST para ejecutar acciones, y almacenar el comprobante en la base de datos local SQLite (Room) para mantener el historial, incluso sin internet.

1. **Depósitos (REST):** El usuario ingresa un monto para recargar su propia cuenta. Suma al saldo total.
2. **Retiros (REST):** El usuario simula un retiro de efectivo. Resta del saldo total (validando fondos suficientes).
3. **Transferencias P2P (REST + WebSocket):** Envío de dinero a otra cuenta. El emisor usa REST para enviar el dinero. El receptor (si tiene la app abierta) recibe la notificación de ingreso en tiempo real a través del WebSocket, sin recargar la pantalla.
4. **Pago de Servicios (Operación Adicional):** Un módulo donde el usuario ingresa un "Código de Contrato" (ej. luz, agua, internet) y paga la factura. Esto genera un registro especial en Room categorizado como "Servicio Básico", con un icono distinto en el historial.

3. Gestión de Hilos y Concurrencia (Evaluación Técnica)

El núcleo de la calificación técnica radica en el manejo de **Corrutinas (Coroutines)** *componentes de un programa informático que permiten pausar y reanudar su ejecución sin bloquear el hilo principal*

- **Aislamiento de Red:** Todas las llamadas REST (OkHttp/Retrofit) y la conexión WebSocket deben despacharse en el hilo de red (Dispatchers.IO).
- **Operaciones I/O:** Las consultas, inserciones y lecturas a la base de datos Room también deben ocurrir en Dispatchers.IO.

- **Sincronización de Interfaz:** Las respuestas del servidor o de la base de datos local deben inyectarse de regreso al hilo principal (Dispatchers.Main) para actualizar el saldo y el historial gráfico sin congelar (bloquear) la aplicación (Application Not Responding - ANR).

4. Descripción de la Interfaz Gráfica (UI)

La interfaz debe ser intuitiva, tipo "Fintech", Las **fintech** (unión de *finance* y *technology*) son empresas y startups que utilizan la innovación tecnológica para ofrecer productos y servicios financieros de forma más rápida, ágil y económica que la banca tradicional. desarrollada con Jetpack (Un Jetpack es un conjunto de herramientas de seguridad, rendimiento y crecimiento) Compose o XML Layouts.

- **Cabecera (Dashboard):**
    - Una tarjeta superior destacada que muestre el **Saldo Actual** en números grandes.
    - Un indicador visual de conexión del WebSocket (ej. un círculo verde o rojo con el texto "En línea" o "Desconectado").
- **Panel de Acciones (Grid de Botones):**
    - Cuatro botones grandes, preferiblemente con iconos: [Depositar], [Retirar], [Transferir], [Pagar Servicios]. Al tocar uno, se debe abrir un cuadro de diálogo (Modal/Dialog) para ingresar el monto y la cuenta/contrato destino.
- **Historial Reactivo (Lista Inferior):**
    - Una lista desplazable (RecyclerView o LazyColumn) que consuma directamente los datos de SQLite (Room).
    - Cada fila debe mostrar: Tipo de operación, fecha/hora, y el monto.
    - **Codificación visual:** Montos en verde para ingresos (Depósitos, Transferencias recibidas) y en rojo para egresos (Retiros, Transferencias enviadas, Pagos de servicios).

5. Modo de Entrega

Los estudiantes deberán presentar un paquete completo que demuestre la funcionalidad en un entorno real o emulado.

1. **Código Fuente:** Enlace a repositorio (GitHub/GitLab) con el proyecto de Android Studio, evidenciando el uso de Room, Retrofit/OkHttp y Corrutinas.
2. **Instalador (APK):** El archivo .apk compilado para su instalación en el dispositivo del evaluador.
3. **Defensa del proyecto 29-10-2026** a través de zoom (si se mantiene la virtualidad). En un lado, el celular (o emulador) mostrando la app. En el otro lado, una terminal o Postman. El estudiante debe demostrar que al enviar dinero desde Postman (simulando a otro usuario), el saldo en la app móvil se actualiza en tiempo real vía WebSocket.
4. **Informe Técnico: Normas APA 7 https://normas-apa.org/wp-content/uploads/Guia-Normas-APA-7ma-edicion.pdf**

5. Rúbrica de Calificación (Sobre 20 puntos)

Criterio de Evaluación	Excelente (4 pts)	Bueno (3 pts)	Regular (2 pts)	Deficiente (0-1 pts)
Persistencia Local (Room)	Crea la BD, entidades y DAOs correctamente. El historial se guarda y sobrevive al reiniciar la app.	Guarda el historial, pero presenta fallos menores al recargar los datos tras reiniciar.	Usa SQLite sin la abstracción de Room, o los datos se borran al cerrar la app.	No implementa base de datos local. Los datos residen solo en memoria volátil.
Integración REST (Emisión)	Implementa correctamente Depósito, Retiro, Transferencia y Pago de Servicios comunicándose con el backend.	Falla en una de las operaciones o no maneja correctamente las respuestas de error HTTP (ej. saldo insuficiente).	La app falla o se cierra inesperadamente al recibir errores del servidor.	No hay conexión REST. Operaciones simuladas localmente.
Tiempo Real (WebSockets)	Mantiene el túnel abierto, escucha transferencias entrantes y actualiza el saldo automáticamente sin recargar.	El WebSocket se conecta, pero requiere que el usuario interactúe para reflejar el nuevo saldo en pantalla.	El socket se desconecta constantemente o la actualización presenta un desfase notable.	No implementa WebSockets (usa Polling) o no actualiza el saldo.
Concurrencia e Hilos	Uso impecable de Corrutinas. Red y BD en Dispatchers.IO, UI en Main. Cero congelamientos (ANR).	Usa hilos secundarios, pero realiza alguna operación ligera de I/O en el hilo principal sin causar bloqueos críticos.	Intenta operaciones de red en el Main Thread causando interrupciones visuales evidentes.	Excepciones NetworkOnMainThreadException que provocan el cierre de la aplicación.
Interfaz (UI) y UX	Diseño moderno, panel de acciones claro, historial con colores condicionales (rojo/verde) y alertas de estado de conexión.	Interfaz funcional pero rudimentaria. Falta algún elemento visual como el estado del socket o colores en el historial.	Interfaz confusa, elementos superpuestos o cuadros de diálogo que no	Interfaz incompleta que no permite ejecutar todas las operaciones requeridas.

			capturan bien los datos.	